

A distributed evolutionary based instance selection algorithm for big data using Apache Spark

Liyang Qin^a, Xiaoli Wang^a, Linzi Yin^{b,*}, Zhaohui Jiang^a

^a School of Automation, Central South University, Changsha 410083, China

^b School of Electronic Information, Central South University, Changsha 410083, China

HIGHLIGHTS

- We proposed a distributed coarse granularity chromosome structure to reduce search space and ensure load balance.
- We proposed a stratified evolution strategy to transform multi-objective to multiple single-objective tasks.
- We proposed a sampling-based fitness function with less time complexity

ARTICLE INFO

Keywords:

Evolutionary algorithm
Instance selection
Apache Spark
Big Data

ABSTRACT

Instance selection is an important preprocessing technology in data mining and machine learning. In this paper, we proposed a novel evolutionary based instance selection algorithm for big data. First, we defined a coarse granularity chromosome structure to reduce the size of search space and costs of chromosome operations (recombination and mutation, etc.). Then a stratified evolution strategy was proposed to remove the hyper parameter in classic fitness function and achieve precise control over the reduction ratio of instances. Finally, a sampling-based fitness function was proposed to reduce the time complexity. Experimental results shown that our new algorithm is efficient to complete the instance selection task on data set with millions of instances in minutes-level. The 10-fold cross-validation also proved that the selection results on many datasets have high nearest neighbor classification accuracy.

1. Introduction

With the development of information technologies, the amount of data generated by human society has increased dramatically. Big data brings many challenges to data mining, such as storage difficulties, low computing efficiency and a lot of noise [1,2], etc. In order to solve these problems, many reduction methods have been proposed to select representative instances or remove redundant information from big data set [3].

As an important method in data reduction, Instance selection algorithm (ISA for convince) has been widely studied and generally divided into two categories, called wrapper methods and filter methods separately [4]. Filter methods only focus on the information hidden in the origin data set, and are usually designed based on some cluster algorithms, such as patterns by ordered projections(POP) [5], pairwise opposite class-nearest neighbor(POC-NN) [6], and the method in [7].

Wrapper methods are applied for a specific classifier to delete some redundant instances, such as generalized condensed nearest neighbor (GCNN) [8] and some evolutionary based instance selection algorithms (EBISA for convince)[9,10]. Moreover, in recent years, many new instance selection algorithms based on machine learning models and other evolutionary algorithms have been proposed, such as [22–24]. The related research works in [4,10,11] has shown that EBISA was better than other classic algorithms both in the reduction rate and the classification accuracy, and the comprehensive performance of heterogeneous recombination and cataclysmic mutation appeared to be best [10].

However, most excellent EBISA [12–17] still encounter many challenges in the big data environment. The related literature analysis shows that there are few effective EBISA reported and tested on datasets with more than millions of instances. We believe that the following reasons caused the dilemma of EBISA in big data environment:

* Corresponding author.

E-mail address: yinlinzi@csu.edu.cn (L. Yin).

<https://doi.org/10.1016/j.asoc.2024.111638>

Received 21 May 2023; Received in revised form 26 February 2024; Accepted 11 April 2024

Available online 19 April 2024

1568-4946/© 2024 Elsevier B.V. All rights reserved.

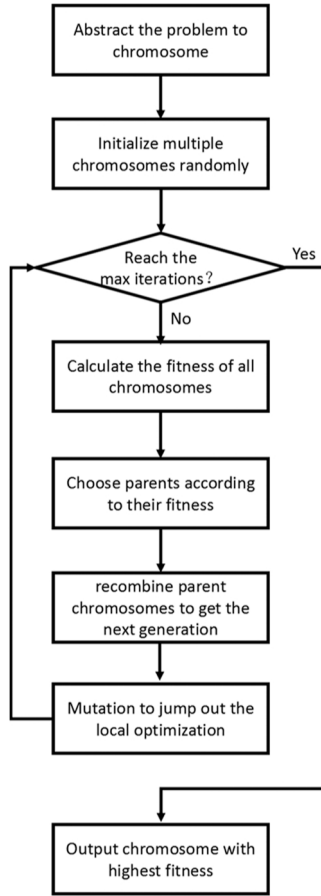


Fig. 1. The structure of the classic evolutionary algorithm.

1.1. Large search space and high dimension of chromosomes

The instance selection task is treated as a combination optimization problem in evolutionary computation, where a chromosome represents a combination of instances and a gene represents an instance. During the evolution process, the chromosomes with better performance (usually refers to higher reduction rate and higher classification accuracy in the nearest neighbor algorithms [10]) are continually split and recombined to generate better solutions. The lengths of chromosomes can easily exceed million-level in big data environment because one gene only represents one instance. Considering the fact that search space would expand dramatically as a chromosome grows [18], the classic EBISA would be difficult to find the optimal solution in big data environment. In addition, these huge chromosomes also cause two main problems:

(1) The time costs of chromosome operations are increased exponentially. For example, assume that the number of iterations is 10, each generation has 10 chromosomes and each recombination operation exchanges half of genes. If there are 1000 instances, then the entire computing process has to exchange 50,000 genes. However, for a data set with 1000,000 instances, the classic EBISA has to exchange 5000,000,000 genes.

(2) The machines are easily out of memory for these huge chromosomes and the massive original big data set. Especially for low dimensional datasets, the size of all chromosomes may be bigger than the original data set. If one uses some serialized methods to store these data to disk, the nature of iterative calculation would generate a large number of disk IO, which would seriously reduce the running efficiency.

1.2. The multi-objective optimization caused by the classic fitness function

The classic fitness function of EBISA is described as follows [12]:

$$fitness(S) = \alpha \bullet acc(S) + (1 - \alpha) \bullet red(S) \quad (1)$$

$acc(S)$ refers to the nearest neighbor classification accuracy of the subset represented by a chromosome and $red(S)$ refers to the reduction rate. In small datasets, this fitness function can achieve high nearest neighbor classification accuracy and reduction rate together. However, in big data environment, it has three main shortcomings:

(1) The high time complexity. Formula (1) calculates the Euclidean distance between any two instances with the time complexity of $O(M \cdot N^2)$, where M is the number of attributes and N is the number of instances related to a chromosome. For example, for a data set with millions of instances, the algorithm has to calculate more than billions Euclidean distances for one chromosome in each iteration.

(2) It is difficult to precisely control the reduction ratio. Formula (1) includes two optimization goals: higher $acc(S)$ and higher $red(S)$ by a hyper parameter α . It is very difficult to precisely control the size of the selection result by adjusting α unless a large number of experiments.

(3) It is difficult to achieve the load balance in parallel computing. One of the most typical parallel acceleration strategies for evolutionary algorithm is to calculate the fitness of each chromosome in parallel. However, the size differences between subsets represented by different chromosomes would be very large in classic EBISA and lead to the data skew in parallel computing. To solve this problem, it is necessary to add complex task assignment strategy.

1.3. Our contributions

To break through the limitations of classic EBISA in big data environment, we put forward a number of targeted optimization strategies to resolve the main challenges faced by classic EBISA. Based on these strategies, we also designed a new distributed evolutionary based instance selection algorithm (DEBISA). The main contributions are as follows:

(1) We proposed a distributed coarse granularity chromosome structure to reduce the size of search space, achieve precisely control over the reduction ratio and ensure the load balance during the parallel computing process of the algorithm.

First, the original data set is divided evenly into several data partitions and each data partition is treated as an independent population. Then each population is also split into several subsets evenly, and each subset is corresponding to a gene. Furthermore, the proportion of 1 and 0 in each chromosome is fixed to be a user-set value to achieve the precise control over the reduction ratio.

Based on the new distributed coarse granularity chromosome structure, all the distributed populations can be processed in parallel, and the chromosome length will be significantly shorter than the original one because each gene represents a subset. These features change the original ultra-high dimensional combination optimization task to several parallel low dimensional combination optimization tasks, which can effectively reduce the size of search space. Furthermore, owing to the same sizes of populations and chromosomes, the new structure avoids the data skew and succeeds the load balance.

(2) We proposed a stratified evolution strategy to transform the multi-objective optimization task in classic EBISA to multiple single-objective optimization tasks, which can effectively increase the proportion of good instances.

The proposed stratified evolution strategy run multiple single-objective evolutionary algorithms one by one to gradually reduce the size of selection result. That is, the number of instances represented by a gene will continually decrease. Besides, owing to the fixed proportion of 1 and 0 in each chromosome, these single-objective evolutionary algorithms remove the hyper parameter α and only calculate nearest

neighbor classification accuracy to improve computational efficiency.

This stratified strategy is effective to improve the proportion of good instance even if the good instances and bad instances were distributed uniformly in each gene.

(3) We proposed a sampling-based fitness function with less time complexity

Although the fitness function of DEBISA only includes the nearest neighbor classification accuracy because of the stratified evolution strategy, the related time complexity is still high. In order to accelerate the calculation on fitness function in big data environment, we proposed a sampling-based fitness function.

In detail, the fitness of a chromosome is defined as the average nearest neighbor classification accuracy of several small subsets that sampled from the chromosome, and the sampling rate is 5–10 % as default. This novel function can effectively reduce the time cost of fitness calculation, and easily realized in parallel.

The subsequent sections of this paper are organized as follows: In the second section, the basic concept of EBISA and Apache Spark will be introduced. In the third section, we will describe all the optimization strategies in DEBISA, then the complete structure of DEBISA will be shown. In the fourth section, we will introduce how to implement DEBISA on Apache Spark and all the engineering optimization details. In the fifth section, all the experimental results will be shown and discussed to prove the effectiveness of DEBISA. The summary and the outlook for future work will be given in the sixth.

2. Preliminary knowledge

2.1. Classic evolutionary based instance selection algorithm

Evolutionary algorithm (also called genetic algorithm) is a bionic optimization algorithm. It abstracts the possible solution to a chromosome, and uses a gene to represent a dimension of the solution. The evolutionary algorithm adopts heuristic methods to select high-quality chromosomes through natural conditions (fitness function) generation by generation, and chromosome recombination, mutation and other operations to generate new and better chromosomes. The workflow of classic evolutionary algorithm is shown in Fig. 1.

Many researchers have proved that evolutionary algorithm can effectively resolve the complex nonlinear equations and combination optimization problems [18], such as instance selection. The classic EBISA is described as follows:

(1) Structure of chromosome: A chromosome has n binary genes and each gene is associated with an instance. If a gene is 1, the associated instance is selected; otherwise the associated instance is removed. **(2) Iterative optimizing method:** In each generation, all chromosomes' fitness, shown as formula (1), are calculated and formed as a roulette (chromosomes with higher fitness will have bigger area on the roulette). Then two chromosomes are picked randomly and some genes are exchanged to generate two new chromosomes. Repeating the picking and exchanging process to generate N chromosomes for making up the new generation. In a very small chance, some genes would be flipped after the recombination completed. The convergence condition includes the threshold of fitness or the maximum number of iterations.

2.2. CHC based instance selection algorithm

CHC, cross generation heterogeneous recombination cataclysmic mutation [19], is an optimized evolutionary algorithm with faster convergence speed and clearer optimization direction. The structure of CHC based instance selection algorithm is as follows:

(1) Structure of chromosome: Same as classic EBISA.

(2) Iterative optimizing method: CHC mix the old and new chromosomes together to select the chromosomes with the highest fitness for the next generation. This strategy ensures that CHC always converge to the better solution space during the entire calculation. Besides, the

recombination process of CHC is limited by the hamming distance to ensure enough differences between new generations and their parents and accelerate the convergence speed of CHC. The mutation of CHC reverses over more than 30 % of genes on some excellent chromosomes to quickly jump out the local search, and would be run only when the algorithm tends to converge.

2.3. MapReduce and Apache Spark

MapReduce is a distributed computing framework, which mainly consists of two basic phases: map and reduce. In map phase, the original data set is first loaded from disk, then each row is converted to a user-set key-value pair, and the computing results are written to disk. In reduce phase, the calculation result of map is loaded from disk, then all rows with same key is aggregated by a user-set reduce function, the related results are also be written to disk. Besides, the shuffle mechanism is used to transmit the scattered data with same key to same machine.

MapReduce provides an excellent solution for distribution computing. But it is not suitable for iterative computing because of some shortcomings, such as disk input and output (IO) costs and network transmission costs, etc. In recent years, Apache Spark has replaced MapReduce as the mainstream solution for big data offline computing. It has high fault tolerance and performs well on the iterative computing tasks because of memory cache, less shuffle and optimizations on disk IO and network transmission. Therefore, Apache Spark is a good implementation tool for the typical iterative algorithm DEBISA.

In Spark, a data set is encapsulated into a read-only data structure called resilient distributed datasets (RDDs) [20]. Data processing operations are divided into two categories:

(1) Transformation, which creates a new RDD based on the existing RDD(s) according to the user-set logic;

(2) Action, which performs some calculation operation on data set, such as counting the number of rows of the RDD.

Spark encapsulates common transformation and action operations into programming interfaces for users. For detailed and complete interface functions and usage methods, please refer to its official documentation. Here we only introduce a few Spark interfaces related to our algorithm.

map(fun): This function is similar with map in MapReduce. It performs the user-set converting function on each row of RDD and converts it to a new data structure.

mapPartitions(fun): This function is similar to map(fun). However, mapPartitions(fun) only performs the converting function on each data partition once. As we all know, performing a function would consume extra time and other resources, so compared to map(fun), mapPartitions is a better choice in many cases.

zipWithIndex: Generate sequential index columns for a data set. For example, for a data set $S = \{x_1, x_2, x_3\}$, one can execute **S.zipWithIndex** and get a new data set $S_{new} = \{(x_1, 1), (x_2, 2), (x_3, 3)\}$

partitionBy(partitioner): Partition a data set according to the user-set partitioner. For example, for a data set with three instance $x_1 = \{1, 2, 3\}$, $x_2 = \{1, 2, 1\}$, $x_3 = \{1, 2, 2\}$, if the logic of partitioner is that the partition index of a instance equals to its last column, then x_1 is assigned to the third partition, x_2 is assigned to the first partition, and x_3 is assigned to the second partition.

filter(fun()): S.filter(fun()) is used to filter some rows from S, and the filtering result is constructed as a new RDD.

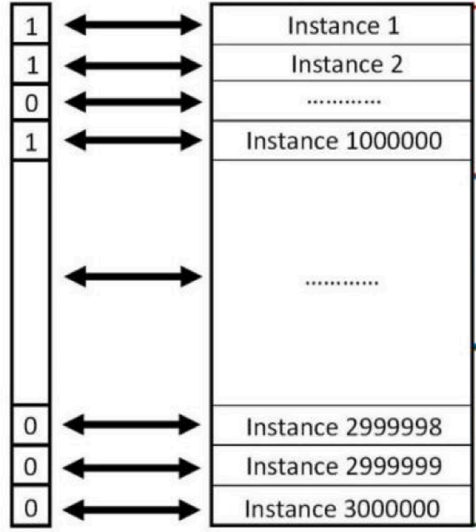
combineByKey(firstMeet, MeetInSamePart, MeetInDiffPart): This function is very similar with the reduce phase in MapReduce, but it is more flexible. Its logic is described as follows:

When executing **KV.combineByKey**, it traverses all partitions of RDD in parallel, includes:

(1) When it first meets a certain key in a partition (this key's container has not been created in this partition), it execute **firstMeet** to create a user-set container for this key to store all the rows with this key in this partition.

Classic chromosome structure:

Each gene represents one instance

**Our new chromosome structure:**

Divide the original dataset into multiple subsets (populations)

Each gene represents multiple instances

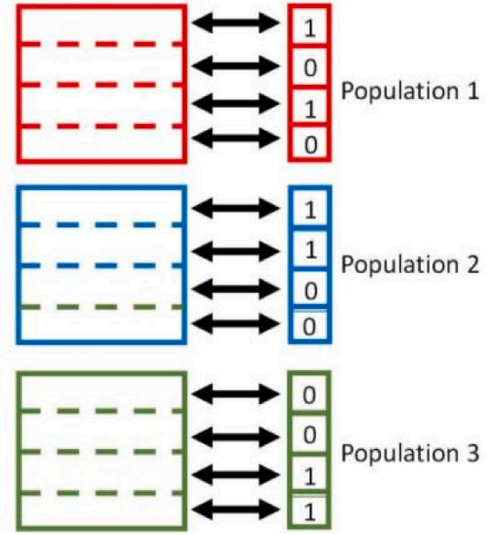


Fig. 2. Our new chromosome structure.

(2) When it meets a row whose key's container in this partition has been created, it executes `MeetInSamePart` to store this row to the corresponding container.

(3) After all partition's `firstMeet` and `MeetInSamePart` completed, `MeetInDiffPart` will be executed to merge the container with same key in different partitions.

sample(put_back, fraction): It is used to randomly sample a subset from an RDD, `put_back` is a Boolean value to control whether put the picked instance back during sampling. **Fraction** is the sampling rate.

3. Distributed evolutionary based instance selection algorithm

In this section, we first introduce the optimization strategies of DEBISA, then present the complete DEBISA.

3.1. The distributed coarse granularity chromosome structure

As discussed in Section 1, the classic chromosome structure leads to a huge search space in big data environment. We apply two main ways to reduce the size of search space. The first one is to divide the original search space into several parallel sub-space, and the other is to change the granularity of each gene to reduce the length of a chromosome. If a gene represents more instances, its granularity is coarser; otherwise, its granularity is finer. For a non-time-series data set, each instance is a record about an independent truth, which means that there is no causality or other correlation between any two instances. Therefore, we suppose that any two genes in a chromosome are independent.

Based on the above analysis, we present a new distributed coarse granularity chromosome structure as follows.

First, the original data set is divided into several independent populations. Considering that EBISA is a computationally intensive task, the number of populations should be related to the number of cores in the cluster. Then, for each population, initialize a group of coarse granularity chromosomes. In detail, all instances in a population are divided

into several disjoint subsets evenly, where one gene represents one subset. Finally, the proportion of 0 and 1 in each chromosome is fixed to be a user-set value forever. That is, the chromosome recombination and mutation operations do not change the proportion of 1 of chromosomes. The advantages of the proposed coarse granularity chromosomes are as follows:

(1) **The reduction rate can be manually set.** For example, if the proportion of 0 in a chromosome is set to 50 %, then half of instances would be deleted in an EBISA. This strategy makes the algorithm can easily achieve precise control over the reduction ratio of instances to suit more scenarios.

(2) **This structure can flexibly adapt to data sets with different sizes.** This coarse granularity structure allows us to specify or limit the length of chromosomes without considering the size of the original data set. That is, no matter how large the data set is, the number of genes in a chromosome does not change. **Benefit from the super short chromosomes,** the time and space costs required for recombination and mutation is effectively reduced. Moreover, the less search space makes the algorithm to find the optimal solution more easily. Furthermore, as the size of data set increases, the increasing of time cost only occurs in the fitness calculation. And the chromosome operations, such as parent selection, recombination and mutation do not generate any extra time costs.

The structure of proposed coarse granularity chromosome and the differences between it and the classic chromosome are shown in Fig. 2.

In the example of Fig. 2, the size of the classical chromosome is 3000,000, while the size of the new chromosome is 4. Thus, the time and space costs required for recombination and mutation is effectively reduced. Of course, the new gene is very coarse, it is necessary to make it finer by iterative EBISA.

3.2. Stratified evolution strategy

In order to improve the proportion of good instances in the selection

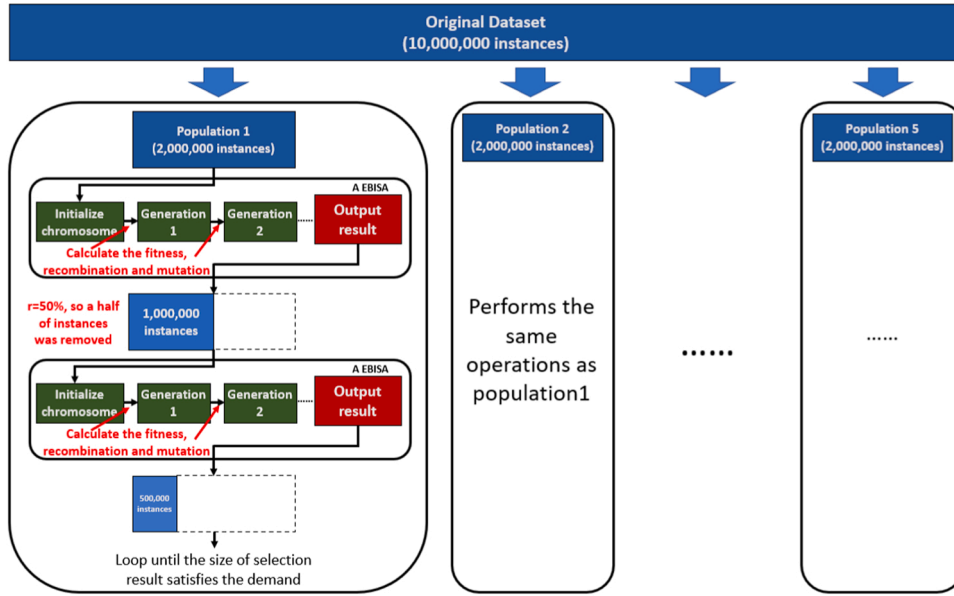


Fig. 3. Stratified evolution strategy.

result, it is necessary to make the coarse granularity gene finer. Therefore, we proposed a stratified evolution strategy, which is described as follows:

(1) For each population, run iterative EBISA t times one by one, and the input of an EBISA is the output of the previous EBISA (the input of the first EBISA is the original data set in this population);

(2) The final result is the union of the output of the last EBISA of all populations.

The iterative number t is defaulted as follows. Assumes that the proportion of 0 in each chromosome is r and the reduction rate of the final selection result is R , then $t = \log_r R$.

A simple example is presented to illustrate the proposed strategy. Assume that the goal of instance selection task is to select 12.5 % good instances from original data set, then we can set $r=50\%$, $t=3$. That is, the proportion of 0 in a chromosome is fixed to be 50 %. The workflow of this strategy is as follows:

(1) Each population independently execute an EBISA in parallel and obtain a selection result S1. Because the proportion of 0 in each chromosome is fixed to be 50 %, the size of S1 is 50 % of the original data set.

(2) Initialize some new chromosomes based on S1. Then start a new EBISA and output S2. Obviously, only 25 % instances of the original data set are remained in S2.

(3) Repeat the step(2) again to remove 50 % instances from the remained instances, then the final result set has 12.5 % instances of the original data set.

This strategy first coarsely selects the good subsets, then find a better result in a more fine-grained way, and repeat it until the required size of result. Obviously, the number of instances represented by a gene in S3 is only 1/3 of that of S1. This strategy has following advantages:

(1) The size of search space in each evolutionary algorithm is small. The length of chromosomes in population is usually short (10 by default), which makes the size of search space in each EBISA to be fixed into $3 \cdot C_{10}^5 = 756$ no matter how big the original data set is. However, the size of search space in classic EBISA can easily reach $C_{10000000}^{1250000}$ when there are more than tens of millions of instances in data set. Compare with the classic EBISA, the proposed DEBISA just needs to process several simple

combination optimization problems in parallel.

(2) This strategy can precisely control the reduction ratio of instances and removes the hyper parameter α . This strategy is easy to predict the reduction rate R by setting r and t , that is, $R = r^t$. Besides, the quality of a chromosome no longer depends on the hyper parameter α that is difficult to explain and normalize. And the common boundary conditions, such as maximizing accuracy under fixed reduction rate, can be easily implemented.

(3) Stratified evolution strategy is effective to improve the quality of result. Even if the good and bad instances are evenly distributed in the original data set, the stratified evolution strategy can constantly change the distribution of instances and granularity of all genes to produce genes that represent more bad instances, and the algorithm is easier to eliminate the chromosomes with such genes to improve the quality of the final selection result.

The workflow of this strategy is shown in Fig. 3.

3.3. A sampling-based fitness function

As discussed in Section 1, the classic fitness function is not suitable for big data computing. Therefore, an efficient fitness function is key to improve the performance of EBISA in big data environment. Owing to the fixed the reduction rate of the coarse granularity chromosome structure, the new fitness function only calculates the nearest neighbor classification accuracy. Therefore, we proposed a sampling-based fitness function to improve the calculation efficiency and the convergence speed.

Definition 1. the fitness of a chromosome CH is defined as the following formula:

$$fitness(CH) = e^{10 \cdot \overline{acc}} \quad (2)$$

Where $\overline{acc} = \sum_{i=1}^k acc_i / k$. acc_i is the nearest neighbor classification accuracy of i th sampling subset, which is randomly sampled $p\%$ instances from the subset represented by chromosome CH, repeat this process k times to get an average $\overline{acc}$.

Because the time complexity of calculating the nearest neighbor classification accuracy is proportional to the square of the number of instances, Therefore, the time complexity of the new fitness is only $kp^2/10000$ of the classical fitness. The exponential function can also accelerate the convergence speed so that the chromosomes with higher classification accuracy are quickly stood out.

3.4. Complete structure of DEBISA

Based on the optimization strategies in Section 3.2, the proposed DEBISA and the related pseudo code are described as Algorithm 1 and the flow chart of DEBISA is shown in Fig. 4.

Algorithm 1. DEBISA

Input: Original dataset **S**
Input: The reduction rate of final selection result **R**
Input: The reduction rate in a single EBISA **r**
Input: The number of independent population **P**
Input: The number of chromosomes in a population **N**
Input: The length of a chromosome **L**
Input: The number of iterations in a complete EBISA **I**
Input: The differences threshold of parents **d**
Output: Selection result

Step 1. Build a sequential index for all instances in **S** // The first instance's index is 0, the second instance's index is 1, and so on.

Step 2. Repartition **S** to **P** partitions according to the index averagely and cache the partitioned dataset in memory. The subset in a partition is treated as an independent population.

Step 3. Initialize the mapping relationship between each gene and all instances and use a hash table **allGeneMapRange** to record this relationship.

Step 4. Heuristic module

Step 4.1 Creating **N** chromosomes in each population randomly and ensure that the proportion of 1 in each chromosome is **r**.

Step 4.2 A complete EBISA (execute independently for each population):

Step 4.2.1 initialize an iteration counter **cnt_in** and set as 1.

Step 4.2.2 Calculate the fitness of each chromosome use formula (2)

Step 4.2.3 Build a roulette, the area of a chromosome in the roulette is equal to its fitness

Step 4.2.4 select two chromosomes that satisfy the hamming distance $\geq L \cdot d$ through the roulette to execute crossover operation, and get two new chromosomes. Exchange some bits in the two new chromosomes to ensure the proportion of 1 in each new chromosome is **r**.

Step 4.2.5 Repeat the step 4.2.4 **N/2** times to generate **N** new chromosomes.

Step 4.2.6 select **N-1** new chromosomes and a best parent chromosome as the next generation.

Step 4.2.7 if **cnt_in** = **t**, jump to Step 4.3, else **cnt_in**=**cnt_in**+1 and jump to Step 4.2.2

Step 4.3 Union the best chromosomes in different populations as **S_current**.

Step 4.4 If the reduction rate of **S_current** has reached reduction rate **R**, output **S_current** as the final selection result, otherwise update **allGeneMapRange** according to the current best chromosomes, includes: remove all instances that are not included by the best chromosomes, re-split all the remained instances averagely to **L** subsets in each population independently, reallocate the mapping relationship between each gene and instances and jump to Step 4.1.

DEBISA on Apache Spark.

4.1. Preprocessing module

In the preprocessing module, we first build an index for each instance and partition the entire data set into several subsets evenly according to the index. Each partitioned subset is treated as an independent population. Next, we design a hash table to record the mapping relationship between gene and instances. We adopted the average partitioning to avoid the data skewness problem that a few machines afford most of the computing task while others are in idle state. Besides, these populations are independent in DEBISA to avoid data transmission among populations and reduce the time costs caused by shuffle.

4. Implementation of DEBISA on Apache Spark

We have open-source all the complete code to Github[21]. One can download it to local computer and run it directly. There are three modules for the implementation of DEBISA on Apache Spark.

(1) **Preprocessing module** implements Step 1, Step 2 and Step 3 in Algorithm 1

(2) **Single EBISA module** implements Step 4.1 to Step 4.3 in Algorithm 1.

(3) **The complete structure of DEBISA** implements Step 4.4 in Algorithm 1, and combine all modules together to build the complete

4.1.1. Build index and partition population

We apply Spark's sequential index generation function **zipWithIndex** to build an index for all the instances in original data set. On the basis, we set a partitioner to divide the original data set to several subsets according to the number of physical cores in the cluster. Next, we convert the partitioned data set to a *key-value* pair RDD, the *key* of each instance is its partition id and *value* is a two-tuple composed of instance and its index. Finally, we cache this *key-value* RDD in memory to avoid repeatedly read it from disk in the subsequent calculating steps. The pseudo code is described in Algorithm 2.

Algorithm 2. Build index and repartition

Input: S, Original dataset
Input: P, The number of partitions
Output: KV1, the key-value pair RDD form of the original dataset, where the key is the partition id and the value is the instance and its index
S_with_index = S.zipWithIndex
S_with_index.partitionBy(partition into P subsets averagely according to the index)
KV1 = S_with_index.mapPartitions: // the workflow of mapPartitions is to execute fun() in each partition in parallel
fun():
 iter = currentPartition.iterator // get the iterator of current partition
 while it.hasNext():
 temp = it.next() // get the current instance and its index
 key = temp.partitionId
 value = (temp.index, temp.intance)
 emit(key, value)
 end while
end fun
end mapPartitions
KV1.persist(MEMORY_AND_DISK) // cache KV1 to memory when the space is enough

4.1.2. Initialize the mapping relationship between genes and instances

In DEBISA, one gene represents a subset, and a hash table is adopted to record the mapping relationship between genes and instances. The implementation of this strategy is described as follows:

(1) For each population, the remained instances are divided into L subsets averagely in order, and a two-tuple is applied to record the start index and the end index of a subset. There is also an array to store all these two-tuples in order.

(2) A hash table called **allGeneMapRange** is created to record the mapping relationship between gene and instances, where the key is the partition id and the value is the array that stores all the index range of all the genes.

A simple example is described to explain this data structure. Assume a data set with 300 samples. It is divided into 3 populations and the length of each chromosome is 4. Then the indexes of the instances in the three populations are {1, 2,, 100}, {101, 102,, 200} and {201, 202,, 300}, respectively. The related hash table **allGeneMapRange** is:

key	value
1	{(1, 25), (26, 50), (51, 75), (76, 100)}
2	{(101, 125), (126, 150), (151, 175), (176, 200)}
3	{(201, 225), (226, 250), (251, 275), (276, 300)}

In this example, each gene represents 25 samples. We will not describe its pseudo code because the logic of this part is very simple and do not need any Spark functions.

4.2. The distributed coarse granularity chromosome structure

DEBISA runs multiple iterative EBISA to calculate the final selection result. In this section, we will show the detail realization on a single EBISA, including chromosomes initialization, a memory caching based parallel subset loading method, the fitness calculation, recombination and mutation.

4.2.1. Chromosomes initialization

In DEBISA, each data partition is treated as an independent population, which contains N chromosomes (10 by default). The length of a chromosome is L (10 by default) and each gene is one or zero. The proportion of 1 in each chromosome is fixed to be a user-set value (default 50 %). In the implementation of DEBISA, all chromosomes are initialized randomly and stored in a hash table, where *key* is the partition id and *value* is a list that contains all the chromosome in this partition. The pseudo code of this part is described in [Algorithm 3](#).

Algorithm 3. Initialize all chromosome and store it to a hash table

Input: P, the number of populations
Output: allChromosomes, a hash table
for partitionId from 0 to P-1:
 current_partition_allChromosomes=new list()
 for chromosomeld from 0 to N-1: // Each population has 10 chromosomes by default
 current_gene=new array(L) // the length of a chromosome is 10 by default
 counter=0
 while counter<L/2 // Randomly select 5 gene and set their value to one
 index=random.nextInt(0, L)
 current_chromosome(index)=1
 counter++
 end while
 current_partition_allChromosomes= current_partition_allChromosomes U current_chromosome
 end for
 allChromosomes=allChromosomes U (partitionId, current_partition_allChromosomes)
end for

4.2.2. A memory caching based parallel subset loading method

In order to calculate the fitness of a chromosome, one has to load the data subset represented by the chromosome. However, the accumulated time costs on repeated disk IO would seriously drag down the running efficiency. Therefore, we proposed a memory caching based parallel subset loading method, which is described as follows:

- (1) Cache all the populations and their index in memory.
- (2) For each population, execute this process in parallel
 - 2.1 select a chromosome.
 - 2.2 Get the gene index ranges of the selected chromosomes.
 - 2.3 Run **filter(fun())** in Spark to remove the instances whose indexes are not recorded in **currentMapRange**.
- (3) Output the union of all subset represented by the selected chromosomes.

An example is also given in Fig. 5. We define a hash table to **allGeneMapRange** to store the index ranges of all genes. The parallel subset loading method is realized as follows.

- (1) select a chromosome from each population.
- (2) In these chromosomes, the index ranges related to all the genes with a value of 1 are read according to the records of **allGeneMapRange**.
- (3) All the instances among the selected index ranges are loaded for the fitness calculation.

This method helps the DEBISA to avoid repeatedly loading huge subsets from disk. Owing to each population selects a subset with very similar size at the same time, the subsequent fitness calculation process will make full use of the computing resources of the cluster and easily achieve load balancing.

The pseudo code of this part is described as Algorithm 4.

Algorithm 4. A memory caching based parallel subset loading method

Input: **KV1**, the cached key-value pair original dataset
Input: **allChromosomes**, a hash table which contains all chromosomes
Input: **allGeneMapRange**, a hash table which records the mapping relationship between genes and instances
Input: **i**, the index of the picked chromosomes
Output: **Si**, an RDD which is composed of all the subsets that corresponding to the *i*th chromosome in each population
 $S_i = KV1.filter$:

```

  currentChromosomes=allChromosomes(partitionId)(i)
  currentMapRange=new list
  // record the instance index that selected by the ith chromosome in this partition
  for (each gene whose value is one) in currentChromosomes:
    index=getCurrentOneIndex
    currentMapRange= currentMapRange ∪ allGeneMapRange(partitionId)(index)
  end for
  // Each instance in each population of KV1 will execute the following fun(), and each population will perform this
  operation independently and in parallel
  fun():
    for each range in currentMapRange:
      // Only instances whose indexes are within the selected range can be retained
      if currentInstance.index>=range.min AND currentInstance.index<=range.max:
        emit(currentInstance) // Save current instance to Si
      end if
    end for
  end fun
end filter

```

4.2.3. The fitness calculation

Here the population-independwe use the **combineByKey** in Spark to implement ent parallel nearest neighbor classification accuracy calculation method. The workflow of **combineByKey** has been introduced in Section 2.3, and three inner functions of **combineByKey** are designed as follows:

(1) **firstMeet**. It initializes a container for each population and puts the first instance of each population to its container. When instance *x* is added to the container, the following information is required:

The label of the instance which is nearest to *x* in the container.

The Euclidean distance between *x* and its nearest instance in the container.

partition id and index that comes with the instance

After **firstMeet** completed, all the instances in the same population would be added to the container one by one. At the same time, **meetInSamePart** would be executed as an instance is added to its container.

(2) **meetInSamePart**. When an instance is added to the container, the Euclidean distance between it and the other instances in the container would be calculated in order and the following operations will be executed:

1. For the instances in the container, if the distance to the new instance is smaller, then adopt the smaller distance and the label of the new instance to update its nearest neighbor classification information (the Euclidean distance to the nearest instance and class of the nearest instance).

2. The new instance sets the nearest instance in the container as its nearest neighbor.

After this process completed, the nearest neighbor classification information(Euclidean distance to the nearest instance and class of the nearest instance) of all instances in the container have been updated.

(3) **meetInDiffPart**. Because each data partition in DEBISA is an independent population, it is not necessary to compare instances in different partitions. Therefore, **meetInDiffPart** is just an empty function without any calculation process.

After all instances have been added to the population's container, we can get the numbers of correctly classified instances and wrongly classified instances of each population. Then we can calculate the nearest

neighbor classification accuracy in each population in parallel. The pseudo codes of **firstMeet** function and **meetInSamePart** function are shown in Algorithm 5 and Algorithm 6 separately. And the workflow of this part is shown in Fig. 6.

Algorithm 5. **firstMeet** function in **combineByKey**

Input: x , the first instance in a partition
Output: **recorder**, a container to record the nearest classification information of each instances in the current population

recorder = new list()
recorder = **recorder** $\cup$ (x , (maxDistance, false)) // when initialize the container, set the distance from x to its nearest instance to infinity, and the classification value is wrong.

Algorithm 6. meetInSamePart function in combineByKey

4.2.4. Recombination and mutation

Owing to the distributed coarse granularity chromosome structure in

Input: **newIns**, an instance that will be added to its container
Input: **recorder**, a container to record the nearest classification information of each instances in the current population
Output: **recorder** with **newIns** contained

newComeRecord=(**newIns**, (maxDistance, false)) //initialize a distance and classification result for **newIns**
for each oldRecord in **recorder**:
 newDistance=calculate_Euclidean_Distance(newComeRecord, oldRecord)
 if newDistance < newComeRecord.distance:
 newComRecord.distance=newDistance
 newComeRecord.classification = (newComRecord.class == oldRecordRecord.class)
 end if

 if newDistance < oldRecord.distance:
 oldRecord.ditance=newDistance
 oldRecord.classification = (newComeRecord.class == oldRecordRecord.class)
 end if
end for
recorder=**recorder** $\cup$ newComeRecord

After **combineByKey** completed, we can get P fitness in parallel. Repeat these processes N times, then the fitness of all chromosomes is calculated. Finally, we use a hash table called **allFitness** to record the all the fitness, where key is the partition id and value is the list which contains all the fitness in this population. The complete fitness calculation process is shown in [Algorithm 7](#).

Algorithm 7. calculate the fitness of all chromosomes

DEBISA, the size of search space is effectively reduced. It is very possible to find a good solution at the beginning of an EBISA. In order to improve the convergence velocity and avoid local optimization, the recombination and mutation operation of DEBISA refers to CHC algorithm and are improved as follows:

(1) Parent chromosomes selection strategy

Owing to the exponential function in the proposed fitness function, the parent chromosomes are easily with high-quality. In order to avoid the algorithm fall into local optimization, we adopt the following parent

Input: **KV1**, the cached key-value pair original dataset
Input: **allChromosomes**, a hash table which contains all chromosomes
Input: **allGeneMapRange**, a hash table which records the mapping relationship between gene and instances
Input: P , the number of populations
Input: L , the length of a chromosome
Input: N , the number of chromosomes in a population
Output: **allFitness**, a hash table which contains the fitness of all chromosomes, where the key is the partition id and the value is a list to store the fitness of all chromosomes in this population.

allFitness = new HashMap
for chromosomeid from 0 to $L-1$:
 subset = algorithm3(input)
 recorder = subset.combineByKey(firstMeet, meetInSamePart, empty)
 for partitionId from 0 to $P-1$:
 currentPartFit = **recorder**.getAllFitnessInSamePartition(partitionId)
 allFitness = **allFitness** $\cup$ (partitionId, currentPartFit)
 end for
end for

chromosomes selection method.

The hamming distance between two parents should satisfy a threshold (default 30 % of the length of a chromosome). Otherwise, DEBISA would re-select two chromosomes (for a recombination, only three re-selections will be perform at most).

This method ensures that there are enough differences between two parent chromosomes and is similar to CHC.

(2) Crossover strategy

DEBISA requires the fixed proportion of 1 in chromosomes. There-

Step 1. Read original dataset S from HDFS
Step 2. Building index for S through **Algorithm 2** and cache it in memory
Step 3. Create the hash table allGeneMapRange which stores the mapping relationship between each gene and all instances through **Algorithm 3**.
Step 4. Heuristic module
 Step 4.1 Create all chromosomes
 Step 4.2 Execute a complete EBISA.
 Step 4.3 reallocate the mapping relationship in allGeneMapRange according to the best chromosome selected by step 4.2
Step 5. When the size of selection result in Step 4.2 satisfies our demand, output the final result according to the best chromosome.

fore, the recombination operation first exchanges several continuous genes from a random position(the length is 30 % of the chromosome by default). Then, some other genes may be randomly exchanged to correct the proportion 1 in the new chromosome as the default value.

(3) The mutation strategy

The mutation operational in DEBISA is similar to that of CHC. That is, it only runs once until the algorithm reached one of the following cases:

1. The number of iterations of the current EBISA reaches three-quarters of the maximum iteration number.
2. The situation that the hamming distance between two parent chromosomes is less than the threshold happens too many times(default 24).

The mutation operation will randomly reverse a quarter of genes in all chromosomes except the best one. After mutation operation completed, the proportion of 1 in each chromosome will also be corrected to the default value.

(4) The next generation of chromosomes strategy

In the classic CHC algorithm, all the parent chromosomes and N new chromosomes will be mixed and sorted according to fitness. Then the best N chromosomes are selected as the next generation. In DEBISA, only the best parent chromosome and N-1 new chromosomes will be remained because the new fitness function of DEBISA has ensured that the algorithm can quickly converge to a small number of high-quality solutions. At the same time, DEBISA avoids the sorting process to improve the calculation efficiency.

The implementation of this part does not need any Spark functions, thus we do not describe its pseudo code.

4.3. The DEBISA based on Spark

Combine all the above modules, we get the main function of complete DEBISA based on Spark, the whole structure is shown in **Algorithm 8**.

In Step 4.3, we use **Algorithm 4** to select the subset represented by the best chromosome as the selection result of this EBISA. If the size of the selection result satisfies the demand, output the result(step 5). Otherwise, delete the indexes that do not selected by the best chromosome and reallocate the mapping relationship between genes and instances in **allGeneMapRange**. The process of the reallocation is described as follows:

Assume that the proportion of 1 in a chromosome is 50 %. After a

EBISA completed, split each index range selected by the best chromosome into two sub-range averagely. Then these sub-ranges compose the **allGeneMapRange** of the next EBISA. Using this method, the search space of the next EBISA is limited in the best subset selected in the previous EBISA. Because the implementation of this part does not need any Spark functions, we will not describe its pseudo code. An example is shown in **Fig. 7**.

Algorithm 8. the main function of DEBISA on Spark

5. Experimental analysis

All experiments are carried out on the supercomputer of the High Performance Computing Center of Central South University. The algorithm uses 40 Intel(R) Xeon(R) Gold 6248 CPUs with 2.5 GHz frequency. The software and their versions involved in the experiments include Hadoop2.7.7, Spark2.3.1, Java1.8.1, Scala2.12.6, Python3.9.2 and Mvapih2-2.3.3.

5.1. Data sets

7 open-source data sets and 4 randomly generated big data sets are shown in **Table 1** and **Table 2**, and the related parameter settings are shown in **Table 3**. All the data sets were performed random out-of-order processing of the whole instances. We adopted the ten fold cross-validation and set the average accuracy of 10 outputs as the nearest neighbor classification accuracy. All the nearest neighbor classification accuracy calculations are obtained through distributed implementation based on MPI4PY.

5.2. Algorithm performance under different reduction rates

In DEBISA, we adopted a stratified evolution strategy to gradually reduce the size of selection result and removed the hyper parameter α . According to the parameter settings in **Table 3**, the running process is described as follows:

- (1) The size of the selection result S1 of the first EBISA is 50 % of the original data set(50 % reduction rate).
- (2) The size of the selection result S2 of the second EBISA is 25 % of the original data set(75 % reduction rate).
- (3) The size of the selection result S3 of the third EBISA is 12.5 % of the original data set(87.5 % reduction rate).

Because of the resources limitations, it is hard to calculate the nearest neighbor classification accuracy on data sets with more than millions of instances when the reduction rate is very low. Therefore, we only tested the performance of DEBISA on Skin, Shuttle, Penbased and Banana in this section.

Fig. 8 shows the relationship between the reduction rate and the nearest neighbor classification accuracy. Although the reduction rate keeps increasing, that is, the size of the selection result keeps decreasing, the classification accuracy of nearest neighbor for the tested set can

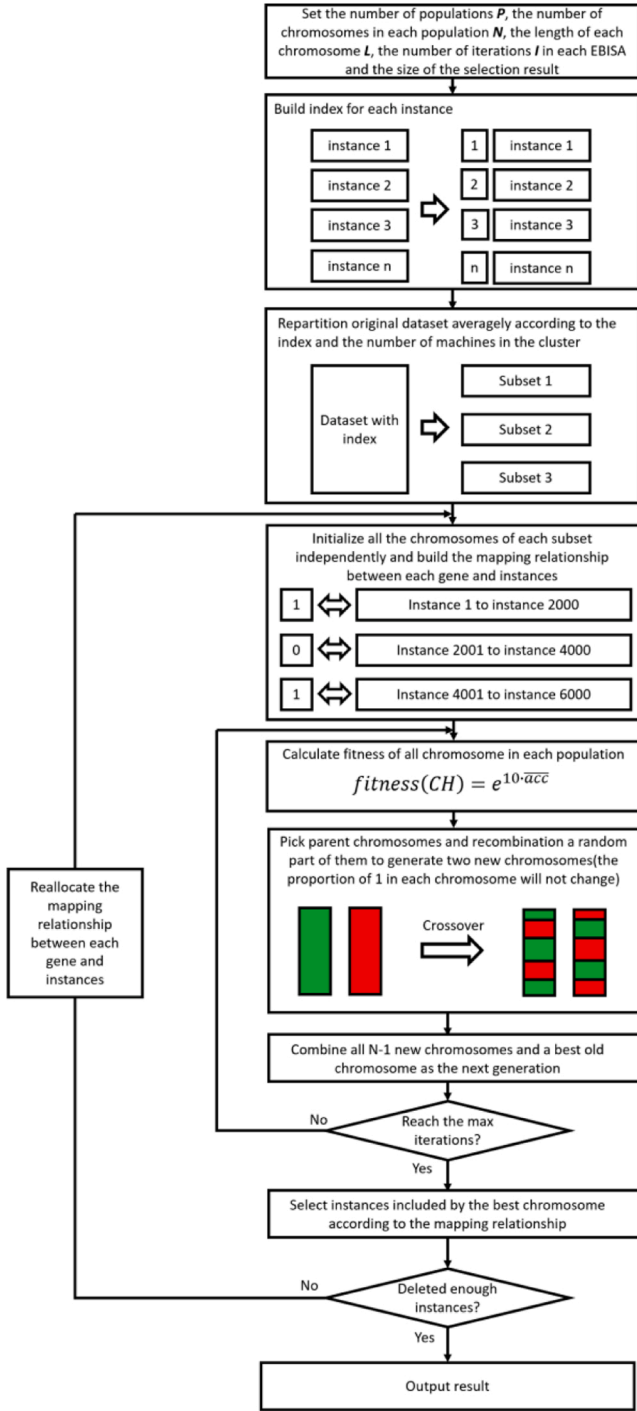


Fig. 4. Structure of DEBISA.

always maintain a high level. Moreover, when the reduction rate increases from 50 % to 87.5 %, the accuracy of nearest neighbor classification on the four datasets still decreases less than 3 %. This experimental result proves that DEBISA can get high quality selection result on big data sets in various reduction rate, which means that DEBISA can be well applied to the selection task of big data.

Fig. 9 shows the relationship between the reduction rate and computation times. The blue column represents the computation time of deleting 50 % of instances from the original data set and get S1. The orange column represents the computation time of further deleting 50 % instances of S1 and get S2. The gray column represents further deleting 50 % instances of S2 and get the final selection result S3. The

experimental result shows that the computation times on Shuttle, Pen-based and Banana are less than 1 min because they are less than 100,000 instances. The calculation time on Skin is still less than 4 min even if it has hundreds of thousands of instances.

Moreover, as the reduction rate increases, the computing time on Skin and Shuttle decreases significantly. This feature means that one can set a high reduction rate to improve the calculation computational efficiency and reduce computation resources. On the other hand, one can also adjust the computing resources dynamically during different steps of DEBISA to improve the utilization of the cluster.

5.3. Algorithm performance under different sampling rates

The new fitness function in formula (2) effectively improves the calculation efficiency and no longer depend on the hyper parameter α which is difficult to be standardized. However, the sampling rate is still set by manual. In order to demonstrate the influence of different sampling rate, we compared the algorithm performance when the sampling is set to be 2.5 %, 5 % and 10 % respectively on HIGGS, SUSY, Poker and Skin data sets with 40 physical cores. The experimental result is show in Fig. 10 and Fig. 11.

Fig. 10 shows the nearest neighbor classification accuracy at different sampling rates. For Higgs and Skin data sets, the algorithm can obtain high-quality instance selection results at all three sampling rates. For Susy and Poker, by increasing the sampling rate from 2.5 % to 10 %, there are more than 12 % improvement in nearest neighbor classification accuracy. According to our analysis, Higgs has only two kinds of attribute values: 0 or 1. And Skin only has 3 attributes and 2 classes. These features means that the distribution of instances in these two data sets may be very simple, and a small sampling subset can accurately represent the whole distribution of the entire data set. However, the attribute value range of Poker and Susy varies greatly, and Poker has 10 classes, which means that the instances distribution of these two data sets may be relatively complex and a small sampling subset is hard to represent the whole distribution of the entire data set.

In Fig. 11, Poker and Skin only increased the total running time about 30 % by increasing the sampling rate from 2.5 % to 10 %. However, the total running time of Susy and Higgs increased nearly 8 times and 5 times, respectively. According to our analysis, due to the small sizes of Poker and Skin (about 100–2000 instances in the sampling subset), the CPU time for logical operation is close to or even less than the time for task switching and resource scheduling. Therefore, increasing the sampling rate in this case can often improve the CPU utilization without significantly increasing the computation time. For Susy and Higgs, the number of instances in each population is about 250,000 and 120,000 respectively. After sampling, the number of instances in sampling subset still reaches 5000–20000, where most of the CPU time has been used for logical calculation. Therefore, there is a significant increase in computation time as the sampling rate increase.

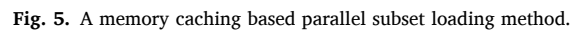
To sum up, when the distribution of data set is simple (with few attributes and classes and small range of attribute values), we can adopt a lower sampling rate such as 2.5 % or 5 % to improve the calculation efficiency. When the instances distribution of the data set is relatively complex or the data quality is low, the sampling rate should be set to 10 % or even bigger.

5.4. Parallel extension performance analysis

We use speedup, scaleup and sizeup to demonstrate the parallel extension performance of DEBISA. Since these three indicators are only used to measure the running speed of the algorithm at different degrees of parallelism, all experiments in this section are carried out on the random big data sets described in Table 2.

5.4.1. Speedup

Speedup is defined as follows:



T_1 represents the runtime of an algorithm running on one node and T_m represents the runtime of the same algorithm running on the same data set and m nodes. If the growth rate of $\text{speedup}(m)$ can approach or

The diagram illustrates the K-Nearest Neighbors (K-NN) algorithm for classification. It shows the step-by-step process of adding a query point to a Recorder and updating the nearest neighbor information.

Initial State:

Feature	Class	
1	1	0
1	0	1
-1	1	0
0	-1	1
-2	1	1

Recorder (Empty)

Step 1: Adding the query point.

Feature	Class	
1	1	0
1	0	1
-1	1	0
0	-1	1
-2	1	1

Recorder:

Feature	Class	Distance	
1	1	0	∞

Nearest neighbor class in Recorder: 1

Nearest neighbor distance in Recorder: ∞

Step 2: Finding the nearest neighbor.

Feature	Class	
1	1	0
1	0	1
-1	1	0
0	-1	1
-2	1	1

Recorder:

Feature	Class	Distance	
1	1	0	1
1	0	1	0

Nearest neighbor: 1

Step 3: Updating the Recorder.

Feature	Class	
1	1	0
1	0	1
-1	1	0
0	-1	1
-2	1	1

Recorder:

Feature	Class	Distance	
1	1	0	1
1	0	1	0
-1	1	0	2
0	-1	1	$\sqrt{2}$
-2	1	1	0

Nearest neighbor: 1

Step 4: Final Recorder state.

Feature	Class	
1	1	0
1	0	1
-1	1	0
0	-1	1
-2	1	1

Recorder:

Feature	Class	Distance	
1	1	0	1
1	0	1	0
-1	1	0	2
0	-1	1	$\sqrt{2}$
-2	1	1	0

Nearest neighbor: 1

12

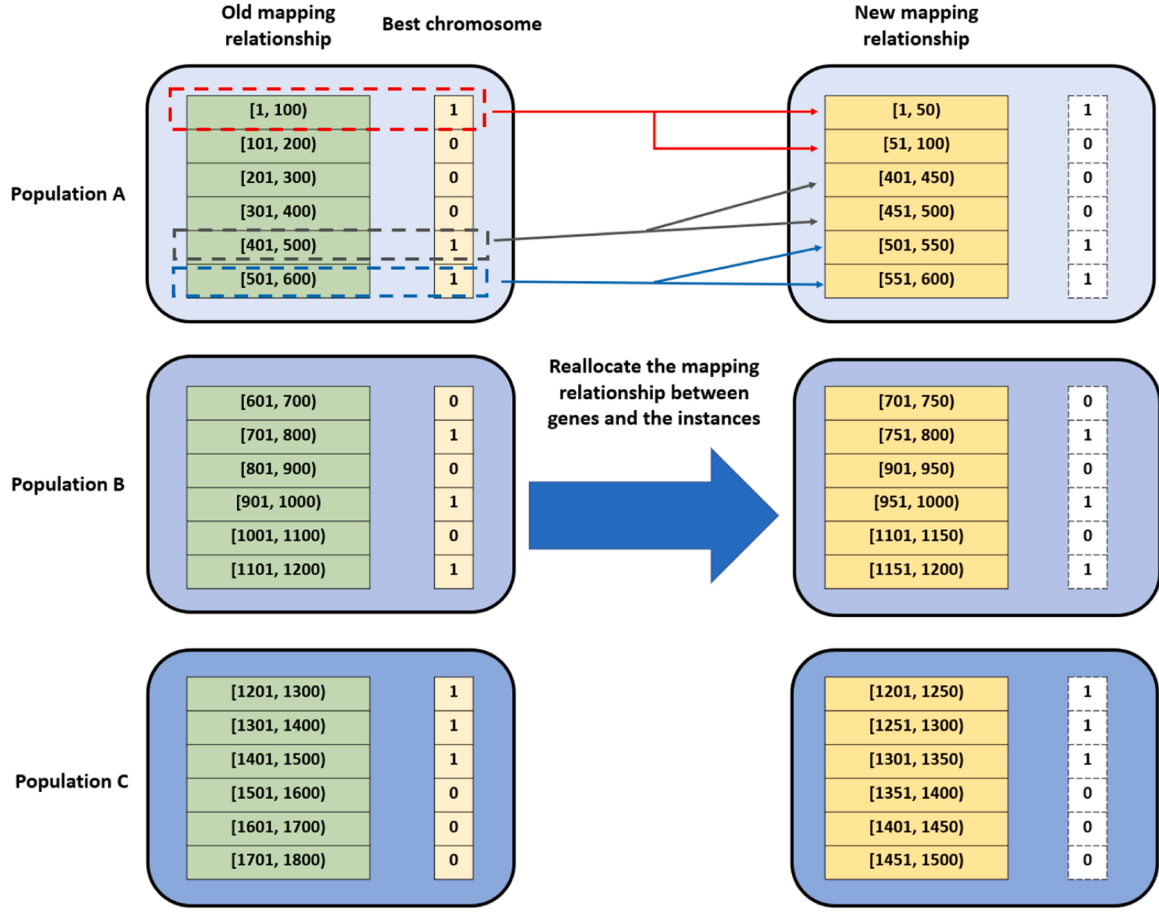


Fig. 7. Reallocating the mapping relationship between genes and instances.

Table 1

Open-source data sets.

Name	Number of instances	Number of attributes	Number of classes
HIGGS[UCI]	11000000	28	2
SUSY[UCI]	5000000	18	2
Poker[keel]	1025010	10	10
Skin[UCI]	245057	3	2
Shuttle[keel]	58000	9	7
Penbased [keel]	10992	16	10
Banana[keel]	5300	2	2

Table 2

Random data sets.

Name	Number of instances	Number of attributes	Number of classes
EA_DS1	5000000	20	2
EA_DS2	4000000	20	2
EA_DS3	3000000	20	2
EA_DS4	2000000	20	2
EA_DS5	1000000	20	2
EA_DS6	500000	20	2

Fig. 12 shows that the speedup performance of EA_DS6 (500,000 instances) was not obvious. However, the speedup growth rate of EA_DS1 (5000,000 instances) exceeds linear increase, which means that DEBISA has an excellent parallel acceleration capability on big data set. This acceleration effect can be explained as follows:

The time complexity of DEBISA's fitness function is proportional to

Table 3

Parameter settings.

Parameter name	Value
Number of iterations of an evolution	10
Reduction rate of an evolution	50 %
Number of evolutions	3
Sampling times of fitness function	4
Sampling rate of fitness function	5 %
Number of populations	0.8×number of cores in cluster
Number of genes in a chromosome	10

the square of the number of instances in the sampling subset. If we increase the number of computing cores in the cluster to P times, then the instances for each core is reduced P times. The computational complexity of each core is thus decreased by P^2 times, which leads to an acceleration effect beyond the linear growth. For EA_DS6, when the cluster has 40 cores, each core only needs to compare the Euclidean distance between hundreds of instances. In this case, the extra time consumption caused by Spark framework may even exceed the algorithm, the acceleration of parallel extension is thus not obviously. To sum up, DEBISA can get obvious acceleration effect with the expanding of the resources in cluster in big data environment.

5.4.2. Scaleup

Scaleup is defined as follows:

$$scaleup(m) = \frac{speedup(m)}{m} \quad (4)$$

m represents the number of nodes and $Speedup(m)$ represents the

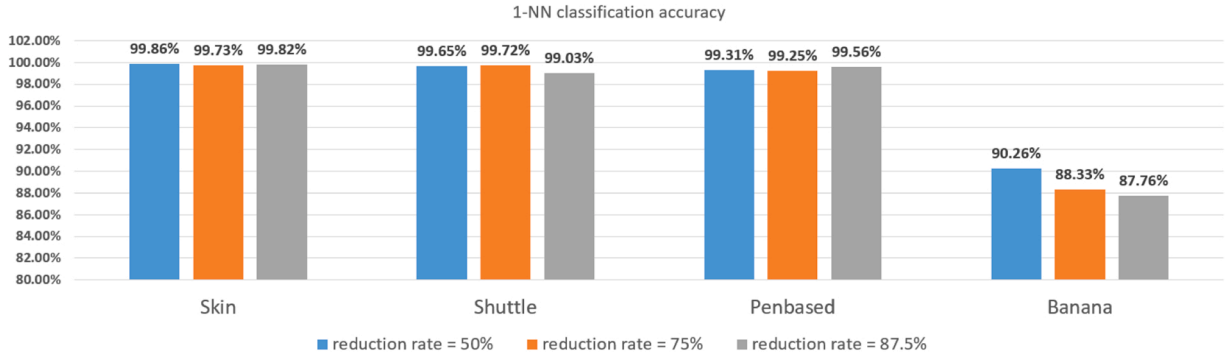


Fig. 8. nearest neighbor classification changes with the reduction rate change.

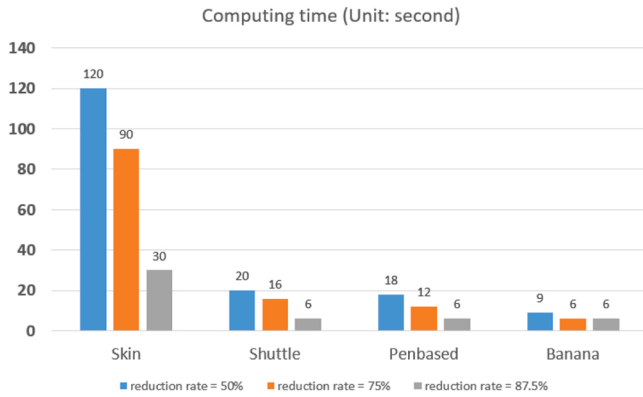


Fig. 9. Computing time of different reduction rate.

speedup with m nodes. If scaleup can be maintained near 1.0 during the synchronous growth of cluster cores and the size of data set, it means that the algorithm can effectively adapt to big data computing of different scales through parallel expansion. We used 10 cores to calculate EA_DS5, 20 cores to calculate EA_DS4, 30 cores to calculate EA_DS3 and 40 cores to calculate EA_DS2 respectively. The scaleup performance is shown in Fig. 13.

Fig. 13 shows that with the synchronous growth of data volume and cluster resources, the scaleup values of DEBISA decreases gently. After the number of cores increases by 4 times, the scaleup value is still more than 0.7, indicating that the algorithm can well adapt to the size change of the data set through parallel expansion. The excellent scaleup performance mainly comes from the strategy of the multiple independent and similar-sized populations according to the number of cores. It has the following advantage:

Suppose the computation ability of each core is same, this strategy can avoid data skewness problem, ensure the load balance and reduce the costs of network transmission. Therefore, when the amount of data grows in proportion to the number of cores, the size of data of each core will not change and the increasing rate of extra time costs of framework will also be very small.

In summary, DEBISA can flexibly adapt to the instance selection tasks of different sizes by adjusting the number of cores in the cluster. This feature means that DEBISA is very suitable for the Cloud computing, HPC and other shared computing environments.

5.4.3. Sizeup

Sizeup is defined as follows:

$$\text{sizeup}(m) = \frac{T_{mDS}}{T_{DS}} \quad (5)$$

T_{DS} represents the runtime of on a data set DS and T_{mDS} represents the runtime of the same algorithm on the data set whose size is m times of that of DS. Sizeup is used to evaluate the time complexity of algorithm. We use 40 cores to calculate the change of sizeup on EA_DS5, EA_DS4, EA_DS3 and EA_DS2 respectively. The experimental result is shown in Fig. 14.

Fig. 14 shows that the growth rate of computing time exceeds linear growth. The main reason is that the time complexity of fitness function is $O(M \cdot S^2)$, where S is the size of the sampling set. But due to the good parallel expansion performance of DEBISA, on the premise of guaranteeing the basic computing resources, DEBISA still has a very high computing efficiency, and can effectively obtain high-quality instance selection results.

5.5. Performance comparison with other EBISA algorithms

In order to further illustrate the superiority of DEBISA, we compared

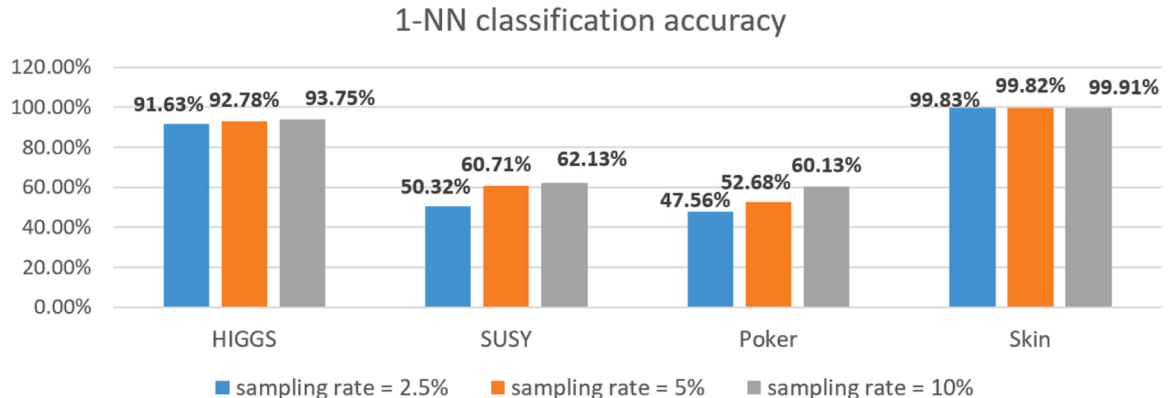


Fig. 10. nearest neighbor classification changes with the sampling rate change.

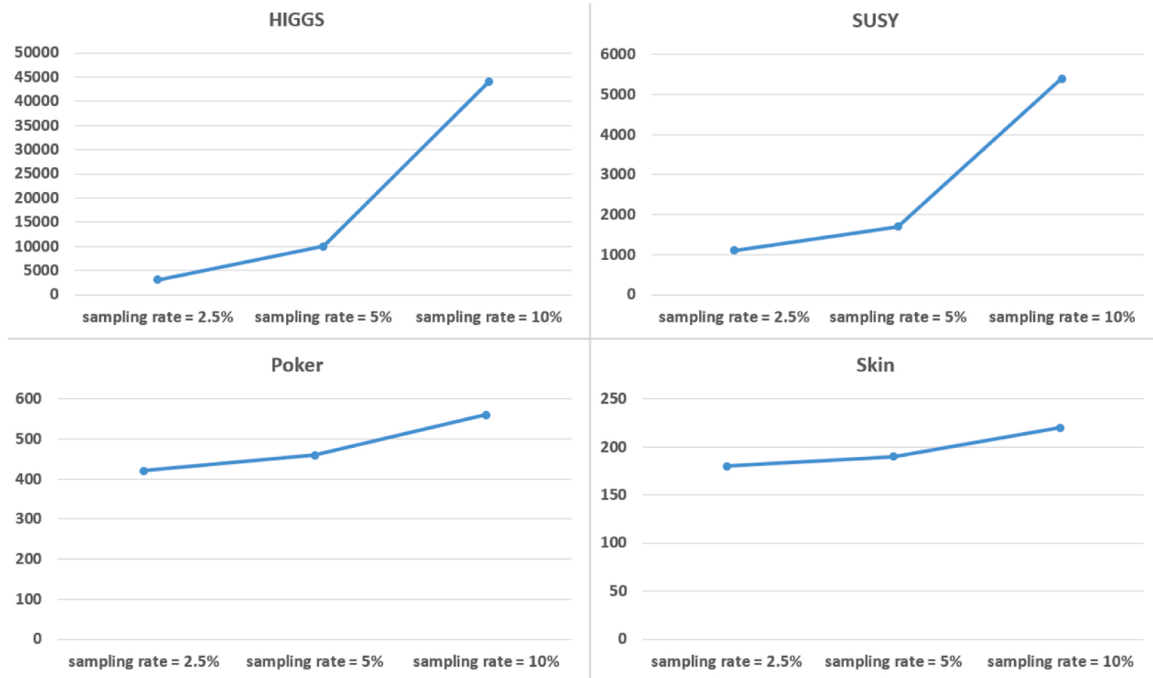


Fig. 11. Computing times with the change of sampling rate (Unit: second).

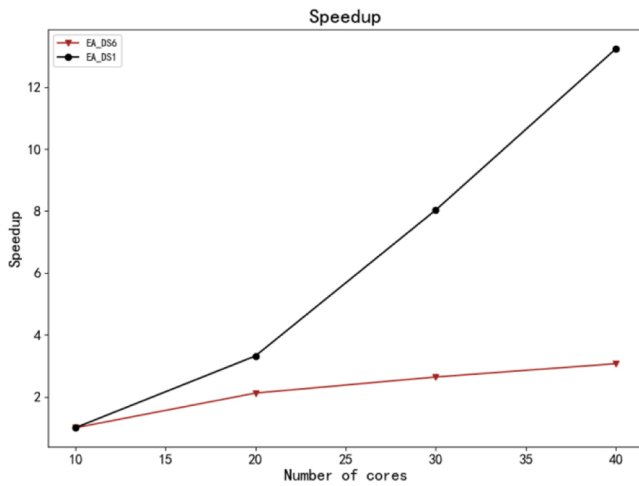


Fig. 12. Speedup.

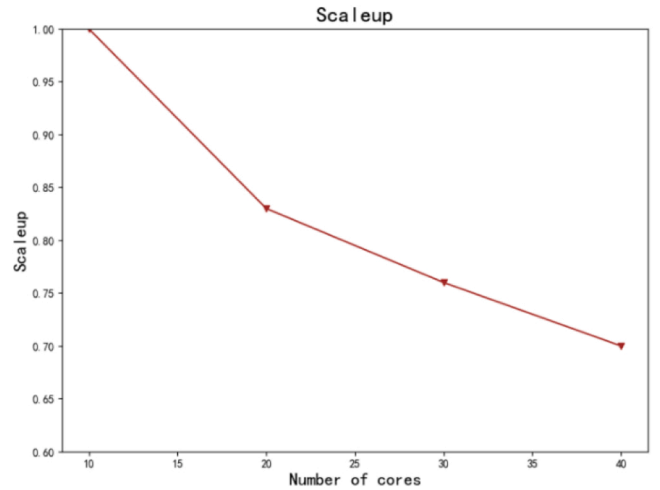


Fig. 13. Scaleup.

the running time and the nearest neighbor classification accuracy of the reduction set on the Banana data set with the representative EBISA EGA [26] and MOCHC [27]. Moreover, we also compared the running time and classification accuracy of the reduction set with LRIS [25], which is a state-of-the-art EBISA for big data set (Because the running time of EGA and MOCHC on the Penbased exceeds 24 h, which means that they do not suitable for such big data set.). To ensure the reliability of the results, we directly used the experimental results of LRIS in its original article [25] for comparison. The comparison results are shown in Table 4.

Table 4 shows that in a distributed computing environment, the computing efficiency of DEBISA is much higher than that of the three classic algorithms, and on the Banana data set, the nearest neighbor classification accuracy of DEBISA's reduction set is also very close to the results of the other three representative algorithms. Moreover, on the high-dimensional big data set Penbased, with similar reduction rates (DEBISA is 87.5 %, LRIS is 84.8 %), the classification accuracy of

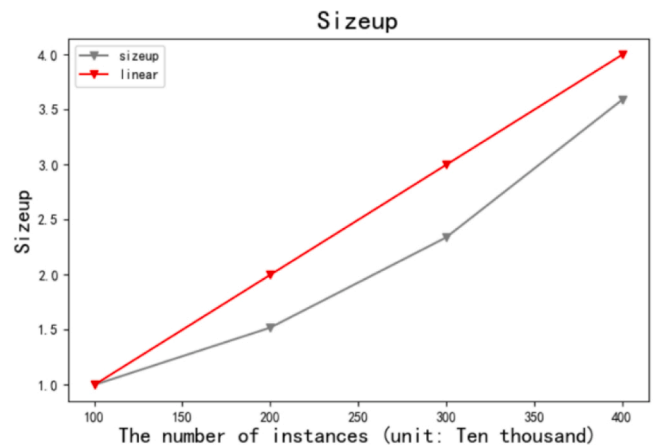


Fig. 14. Sizeup.

Table 4
Comparison results.

	DEBISA		LRIS		EGA		MOCHC	
	Accuracy	Running time	Accuracy	Running time	Accuracy	Running time	Accuracy	Running time
Banana	87.76 %	21 s	89.70 %	1065 s	88.61 %	16137 s	89.01 %	18101 s
Penbased	99.56 %	36 s	98.49 %	16425 s	×	×	×	×

DEBISA is slightly higher than LRIS.

6. Conclusion

DEBISA breaks through many limitations of classic EBISA by some new technologies, such as the coarse granularity chromosome structure, the stratified evolution strategy, and the sampling-based fitness function, etc. At the same time, we present some big data optimal ways based on Apache Spark, such as the hash table to record the mapping relationship between gene and instances, the average partitioning to avoid the data skewness problem and reduce data transmission, the memory caching based parallel subset loading method, the population-independent parallel nearest neighbor classification accuracy calculation method based on combineByKey function, etc. DEBISA has excellent computational efficiency to complete instance selection task on data set with millions of instances in minutes-level, and the nearest neighbor classification accuracy of DEBISA's selection results is more than 90 % on several big data sets. In addition, the experiment result on speedup, scaleup and sizeup showed that DEBISA has excellent parallel expansion performance, which means that DEBISA is suitable for cloud computing, HPC and other shared computing environments.

However, DEBISA still has the following two major limitations: (1) Because DEBISA adopts a coarse-grained instance selection strategy, although it improves efficiency, it may lose some low-proportion but relatively important parts in the selection process when facing a data set with significant unbalanced instance distribution. (2) DEBISA's multiple iteration selection strategy is not suitable for processing streaming data sets, and the problem of unbalanced instance distribution is often more serious in streaming data sets. However, in many practical application scenarios, unbalanced streaming data sets often occupy a major part of many actual tasks. Therefore, it is worth as the next research direction to improve EBISA and apply it to the instance selection task of unbalanced big data sets.

Funding

This work was supported by the Natural Science Foundation of China under Grant 61773406 and the Natural Science Foundation of Hunan Province China under Grant 2021JJ30877.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data Availability

The data that has been used is confidential.

References

- [1] H.N. Dai, H. Wang, G.Q. Xu, J.F. Wan, M. Imran, Big data analytics for manufacturing internet of things: opportunities, challenges and enabling technologies, *Enterp. Inf. Syst.* 14 (9-10) (2020) 1279–1303.

- [2] M. Chen, S.W. Mao, Y.H. Liu, Big data: a survey, *Mob. Netw. Appl.* 19 (2) (2014) 171–209.
- [3] S. Li, N. Marsaglia, C. Garth, J. Woodring, J. Woodring, J. Clyne, H. Childs, Data reduction techniques for simulation, visualization and data analysis, *Comput. Graph. Forum* 37 (6) (2018) 422–447.
- [4] J.A. Olvera-Lopez, J.A. Carrasco-Ochoa, J.F. Martinez-Trinidad, J. Kittler, A review of instance selection methods, *Artif. Intell. Rev.* 34 (2) (2010) 133–143.
- [5] J.C. Riquelme, J.S. Aguilar-Ruiz, M. Toro, Finding representative patterns with ordered projections, *Pattern Recognit.* 36 (4) (2003) 1009–1018.
- [6] T. Raicharoen, A divide-and-conquer approach to the pairwise opposite class-nearest neighbor (POC-NN) algorithm, *Pattern Recognit.* 26 (10) (2005) 1554–1567.
- [7] A. Lumini, L. Nanni, A clustering method for automatic biometric template selection, *Pattern Recognit.* 39 (3) (2006) 495–497.
- [8] C.H. Chou, B.H. Kuo, F. Chang, The generalized condensed nearest neighbor rule as a data reduction method, *Proc. 18th International Conference on Pattern Recognition*, Hongkong, PR China, 2006, p. 556.
- [9] J. Derrac, C. Cornelis, S. García, F. Herrera, Enhancing evolutionary instance selection algorithms by means of fuzzy rough set based feature selection, *Inf. Sci.* 186 (2012) 73–92.
- [10] J.R. Cano, F. Herrera, M. Lozano, Using evolutionary algorithms as instance selection for data reduction in KDD: an experimental study, *IEEE Trans. Evolut. Comput.* 7 (6) (2003) 561–575.
- [11] P.R. Javier, A.P. Alexis Germán, G.P. Nicolás, Simultaneous instance and feature selection and weighting using evolutionary computation: proposal and study, *Appl. Soft Comput.* 37 (3) (2015) 416–443.
- [12] A. Rosales-Perez, S. García, J.A. González, C.A.C. Coello, F. Herrera, An evolutionary multiobjective model and instance selection for support vector machines with pareto-based ensembles, *Trans. Evolut. Comput.* 21 (6) (2017) 863–877.
- [13] M. Antonelli, P. Ducange, F. Marcelloni, Genetic training instance selection in multiobjective evolutionary fuzzy systems: a coevolutionary approach, *IEEE Trans. Fuzzy Syst.* 20 (2) (2012) 276–290.
- [14] N. Verbiest, S. Vluymans, C. Cornelis, N. Garcia-Pedrajas, N. Y. Saeys, Improving nearest neighbor classification using ensembles of evolutionary generated prototype subsets, *Appl. Soft Comput.* 44 (2016) 75–88.
- [15] Z. Chen, W. Lin, S. Ke, Evolutionary feature and instance selection for traffic sign recognition, *Comput. Ind.* 74 (SI) (2015) 201–211.
- [16] F. Cheng, F.X. Chu, L. Zhang, A multi-objective evolutionary algorithm based on length reduction for large-scale instance selection, *Inf. Sci.* 576 (2021) 105–121.
- [17] N. Garcia-Perajas, J.A.R. del Castillo, D. Oriz-Boyer, A cooperative coevolutionary algorithm for instance selection for instance-based learning, *Mach. Learn.* 78 (3) (2010) 381–420.
- [18] J.R. Cano, F. Herrera, M. Lozano, Evolutionary stratified training set selection for extracting classification rules with trade off precision-interpretability, *Data Knowl. Eng.* vol. 60 (1) (2007) 90–108.
- [19] L.J. Eshelman, Synthetic structure of industrial plastics, in *Foundation of Genetic Algorithms*, 1st ed. New York, USA: Elsevier, pp. 265–283.
- [20] L.Z. Yin, L.Y. Qin, Z.H. Jiang, X.M. Xue, A fast parallel attribute reduction algorithm using Apache Spark, *Knowl.-Based Syst.* 212 (2021) 1–22.
- [21] Qin (2021) DEBISA (Version 0.1) [Source code]. <https://github.com/BigYellowTiger/DEBISA>.
- [22] H.Y. Pan, S.X. Chen, H.L. Xiong, A high-dimensional feature selection method based on modified Gray Wolf Optimization, *Appl. Soft Comput.* 135 (2023) 110031.
- [23] C.C. Lin, J.R. Kang, Y.L. Liang, C.C. Kuo, Simultaneous feature and instance selection in big noisy data using memetic variable neighborhood search, *Appl. Soft Comput.* 112 (2021) 107855.
- [24] M. Aslani, S. Seipel, A fast instance selection method for support vector machines in building extraction, *Appl. Soft Comput.* 97 (2020) 106716.
- [25] F. Cheng, F.X. Chu, L. Zhang, A multi-objective evolutionary algorithm based on length reduction for large-scale instance selection, *Inf. Sci.* 576 (2021) 105–121.
- [26] C.F. Tsai, Z.Y. Chen, Towards high dimensional instance selection: an evolutionary approach, *Decis. Support Syst.* 61 (2014) 79–92.
- [27] S. Rathee, S. Ratnoo, J. Ahuja, Instance selection using multi-objective CHC evolutionary algorithm, *Information and Communication Technology for Competitive Strategies*, 40, pp. 475–484, 2019.